



Docker

مقدمه

هدف از این پروژه، آشنایی با ابزار داکر (Docker) است. شما در ابتدا باید بخش بک‌اند و فرانت‌اند پروژه خود را Dockerize کرده و سپس از ابزار Docker Compose استفاده کنید تا به راحتی بک‌اند و فرانت‌اند خود را به همراه پایگاه‌داده آن بالا بیاورید.

Docker

ابزار داکر به ما این امکان را می‌دهد که برنامه‌ها را به صورت مجزا و در محیطی کاملاً ایزوله راه‌اندازی کنیم. به این محیط ایزوله، کانتینر (Container) می‌گویند. کانتینرها این امکان را برای توسعه‌دهندگان فراهم می‌کنند که یک برنامه را به همراه تمام مازول‌ها و وابستگی‌های آن (مانند کتابخانه‌ها) در کنار هم قرار داده و در قالب یک پکیج آماده اجرا، ارائه کنند. این پکیج می‌تواند در سیستم‌های مختلف بدون نیاز به نصب مجدد نیازمندی‌ها و وابستگی‌ها اجرا شود. بنابراین با وجود کانتینر یک برنامه، دیگر نگران تنظیمات و نیازمندی‌های آن نیستیم و می‌توانیم آن را به راحتی اجرا کنیم.

در واقع، کانتینر یک نمونه اجرا شده از یک Image است. هر Image یک پکیج غیر قابل تغییر از یک برنامه است که اطلاعات و دستورات مورد نیاز برای اجرای یک کانتینر از آن را نگهداری می‌کند. جهت ساخت یک Image برای برنامه خود، از Dockerfile استفاده می‌کنیم. Dockerfile یک اسکریپت است که دستورات آن نحوه ساخت Image ما را مشخص می‌کنند. ما در یک Dockerfile، معمولاً از یک Image پایه شروع می‌کنیم و کارهای مورد نیاز جهت راه‌اندازی برنامه خود در آن را به ترتیب انجام می‌دهیم.

جهت آشنایی با داکر این [لینک](#) را مطالعه کنید. داکر را روی سیستم خود نصب کنید و با مفاهیم Image و Container و همچنین مزیت‌های داکر نسبت به یک VM را بررسی کنید.

Frontend Dockerization

تا اینجا احتمالاً بخش فرانت‌اند پروژه‌ی خود را در حالت توسعه (Development Mode) اجرا می‌کردید؛ یعنی با استفاده از ابزارهایی مانند `npm start`، `npm run dev` یا سرور توسعه‌ی React. این روش برای توسعه مناسب است، اما برای محیط عملیاتی (Production) مناسب نیست. در محیط Production، باید ابتدا پروژه‌ی React خود را build کنید تا فایل‌های نهایی و بهینه‌شده‌ی استاتیک تولید شوند. سپس این فایل‌ها را با استفاده از یک static file server مناسب، مانند Nginx، در اختیار کاربران قرار دهید. برای داکرایز کردن فرانت‌اند، باید از Multi-stage Docker build استفاده کنید. در مرحله‌ی اول، پروژه‌ی فرانت‌اند را build کنید و در مرحله‌ی دوم، فایل‌های build شده را داخل یک image مبتنی بر Nginx قرار دهید و از طریق آن سرو کنید.

Reverse Proxy

در این پروژه لازم است درباره‌ی مفهوم Reverse Proxy تحقیق کنید، مزایا و هزینه‌ها/معایب آن را بررسی کنید، و سپس Nginx را به‌گونه‌ای تنظیم کنید که به عنوان reverse proxy بین فرانت‌اند و بک‌اند عمل کند.

Backend Dockerization

تاکنون برای اجرای بخش بک‌اند پروژه خود، احتمالاً وابستگی‌ها و محیط‌های مورد نیاز (مانند مفسرها، کامپایلرها، کتابخانه‌ها و پکیج‌مدیرها) را مستقیماً روی سیستم عامل خود نصب و اجرا می‌کردید. چالش اصلی این روش، عدم یکپارچگی محیط‌های اجرا بین سیستم توسعه‌دهندگان مختلف و در نهایت سرور عملیاتی (Production) است. هدف داکرایز کردن بک‌اند، بسته‌بندی کل برنامه به همراه تمام پیش‌نیازهای محیطی آن در یک کانتینر ایزوله و مستقل است.

برخلاف فرانت‌اند که فایل‌های استاتیک آن را کامپایل کرده و به یک وب‌سرور خارجی (مثل Nginx) می‌سپاریم، در بک‌اند به محیطی نیاز داریم که کد پویای ما را به طور مداوم اجرا کرده و به درخواست‌های دریافتی پاسخ دهد.

برای داکرایز کردن بک‌اند باید از یک ساختار Multi-stage Docker build استفاده کنید. در مرحله اول، پروژه‌ی جاوای خود را با استفاده از ابزار ساخت مناسب کامپایل کرده و فایل اجرایی نهایی را تولید کنید. در مرحله دوم، تنها فایل اجرایی ساخته شده را داخل یک image سبک مبتنی بر JRE قرار دهید و از طریق آن سرویس بک‌اند را اجرا کنید.

Docker Compose

ابزار داکر کامپوز (Docker Compose) یک ابزار Container Orchestration است. این ابزار به ما این امکان را می‌دهد که چندین کانتینر مرتبط را به صورت همزمان و هماهنگ اجرا کنیم. به طور مثال، به جای اینکه با یک دستور کانتینر بک‌اند و با یک دستور دیگر کانتینر فرانت‌اند را بسازیم، می‌توانیم با نوشتن یک فایل compose.yaml هماهنگی بین همه کانتینرهای خود را تعریف کنیم و سپس با `docker compose up` آن‌ها را بالا بیاوریم.

داخل فایل متنی `compose.yaml` (که در نسخه‌های قدیمی‌تر `docker-compose.yaml` بود و با استفاده از برنامه `docker-compose` کار می‌کرد)، نحوه راه‌اندازی و پیکربندی هر یک از کانتینرها تعریف می‌شود. برای مثال، می‌توان مشخص کرد که هر سرویس از چه Image ای استفاده می‌کند، چه port هایی دارد،

مقدار تنظیم شده برای environment variable های آن چیست، ترتیب اجرای سرویس ها چگونه باشد و هر کدام چه network ها و volume هایی دارند.

برای آشنایی بیشتر با Compose، این [لینک](#) را مطالعه کنید. با مفاهیم networking و volume ها (دو نوع named volume و host/bind mount) در داکر آشنا شوید.

دو کانتینر که روی یک network قرار دارند، همدیگر را می شناسند (به طور کامل از هم ایزوله نیستند) و اسم کانتینر آن ها به IP آن کانتینر resolve می شود. بنابراین یک کانتینر می تواند در صورت قرار گرفتن در network یکسان، مثلاً به آدرس http://some-container:8080 ریکوئست ارسال کند.

volume ها در داکر می گذارند که حافظه ای ثابت برای یک کانتینر وجود داشته باشد. این یعنی در صورت ساخته شدن مجدد آن کانتینر، بخشی از حافظه آن حذف نشده و باقی بماند. volume ها باید به یک فولدر در داخل کانتینر mount شوند. برای این کار یا صرفاً یک named volume را mount می کنیم (که حافظه دائم پشت آن را خود داکر هندل می کند) و یا یک bind mount می سازیم که یک فولدر در سیستم خودمان را به یک فولدر در کانتینر می پیونداند.

Database Dockerization

برای داکرایز کردن پایگاه داده، نیازی به ساخت Dockerfile جدید نیست. کافی است از ایمج رسمی پایگاه داده ای که در پروژه استفاده می کنید استفاده کرده و آن را از طریق Environment Variables (متغیرهای محیطی) پیکربندی کنید.

با توجه به اینکه کانتینرها به طور پیش فرض موقتی (Ephemeral) هستند و با حذف کانتینر، داده های داخل آن نیز از بین می روند، شما باید حتماً از Docker Volumes استفاده کنید تا داده های پایگاه داده را روی سیستم میزبان (Host) ذخیره و پایدار (Persist) کنید.

نکات پایانی

- این پروژه به صورت انفرادی است.
- برای تحویل پروژه کد های خود را zip و در صفحه درس بارگذاری کنید.
- تسلط به مفاهیم docker مهم بوده و در تحویل از آنها سوال پرسیده خواهد شد.
- انتخاب base image های مناسب برای stage های مختلفی که دارید مهم است و باید برای انتخاب خود دلیل قانع کننده داشته باشید.
- حفاظت از مقادیر پیکربندی حساس و secret ها حائز اهمیت بوده و باید تدابیر امنیتی مناسبی را برای حفاظت از آنها در نظر بگیرید. (به ساده ترین شکل ممکن امنیت آنها را تامین کنید)
- هدف این تمرین یادگیری و دور کردن ذهن شما از شرایط فعلی است. لطفا تمرین را خودتان انجام دهید.